

1. Structure of a C Program

A C program follows a fixed layout: preprocessor includes, global declarations, function definitions, and main().

```
#include <stdio.h>
#include <string.h>
#define MAX 100
int globalVar = 0;
void myFunction(int x)
{
    printf("%d\n", x);
}
int main()
{
    int n;
    scanf("%d", &n);
    myFunction(n);
    return 0;
}
```

Key components: #include (headers), #define (macros), global variables, function definitions, main() entry point, return 0.

2. If-Else Problems

Problem 2.1 — Odd or Even

Given an integer N , print 'Even' if divisible by 2, else print 'Odd'.

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    if (n % 2 == 0)
    {
        printf("Even\n");
    }
    else
    {
        printf("Odd\n");
    }
    return 0;
}
```

Time Complexity	$O(1)$
Space Complexity	$O(1)$

Problem 2.2 — Grade Classifier (Nested If-Else)

Given marks (0–100), print: $A+(\geq 90)$, $A(\geq 80)$, $B(\geq 70)$, $C(\geq 60)$, $F(< 60)$. Print 'Invalid' if out of range.

```
#include <stdio.h>
int main()
{
    int m;
    scanf("%d", &m);
    if (m >= 0 && m <= 100)
    {
        if (m >= 90) { printf("A+\n"); }
        else if (m >= 80) { printf("A\n"); }
        else if (m >= 70) { printf("B\n"); }
        else if (m >= 60) { printf("C\n"); }
        else { printf("F\n"); }
    }
    else { printf("Invalid\n"); }
    return 0;
}
```

Time Complexity	$O(1)$
Space Complexity	$O(1)$

3. Loop Problems (While Loop)

Problem 3.1 — Sum of Digits

Given a non-negative integer N , find the sum of its digits.

```
#include <stdio.h>
int main()
{
    int n, sum = 0;
    scanf("%d", &n);
    while (n > 0)
    {
        sum += n % 10;
        n /= 10;
    }
    printf("%d\n", sum);
    return 0;
}
```

Time Complexity	$O(\log N)$
Space Complexity	$O(1)$

Problem 3.2 — Multiplication Table (Nested While)

Given N , print the $N \times N$ multiplication table.

```
#include <stdio.h>
int main()
{

```

```
int n, i = 1, j;
scanf("%d", &n);
while (i <= n)
{
    j = 1;
    while (j <= n)
    {
        printf("%4d", i * j);
        j++;
    }
    printf("\n");
    i++;
}
return 0;
}
```

Time Complexity	$O(N^2)$
Space Complexity	$O(1)$

4. Array Problems

Problem 4.1 — Find Max and Min

Given an array of N integers, print the maximum and minimum elements.

```
#include <stdio.h>
int main()
{
    int n, i, mx, mn;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    mx = mn = a[0];
    for (i = 1; i < n; i++)
    {
        if (a[i] > mx) mx = a[i];
        if (a[i] < mn) mn = a[i];
    }
    printf("Max=%d Min=%d\n", mx, mn);
    return 0;
}
```

Time Complexity	$O(N)$
Space Complexity	$O(N)$

Problem 4.2 — Reverse an Array In-Place

Given an array of N integers, reverse it in-place and print the result.

```
#include <stdio.h>
int main()
{
    int n, i, tmp;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    int lo = 0, hi = n - 1;
    while (lo < hi)
    {
        tmp = a[lo]; a[lo] = a[hi]; a[hi] = tmp;
        lo++; hi--;
    }
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    return 0;
}
```

Time Complexity	O(N)
Space Complexity	O(N)

5. Binary Search

Problem 5.1 — Classic Binary Search

Given a sorted array of N integers and a key, return its index or -1 if not found.

```
#include <stdio.h>
int binarySearch(int a[], int n, int key)
{
    int lo = 0, hi = n - 1, mid;
    while (lo <= hi)
    {
        mid = lo + (hi - lo) / 2;
        if (a[mid] == key) return mid;
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
int main()
{
    int n, key, i;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &key);
    printf("%d\n", binarySearch(a, n, key));
    return 0;
}
```

Time Complexity	O(log N)
Space Complexity	O(1)

Problem 5.2 — Lower Bound

Given a sorted array and key, find the first index i where $a[i] \geq \text{key}$.

```
#include <stdio.h>
int lowerBound(int a[], int n, int key)
{
    int lo = 0, hi = n, mid;
    while (lo < hi)
    {
        mid = lo + (hi - lo) / 2;
        if (a[mid] < key) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
int main()
{
    int n, key, i;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &key);
    printf("%d\n", lowerBound(a, n, key));
    return 0;
}
```

Time Complexity	O(log N)
Space Complexity	O(1)

Problem 5.3 — Upper Bound

Given a sorted array and key, find the first index i where $a[i] > \text{key}$.

```
#include <stdio.h>
int upperBound(int a[], int n, int key)
{
    int lo = 0, hi = n, mid;
    while (lo < hi)
    {
        mid = lo + (hi - lo) / 2;
        if (a[mid] <= key) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

```
int main()
{
    int n, key, i;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    scanf("%d", &key);
    printf("%d\n", upperBound(a, n, key));
    return 0;
}
```

Time Complexity	O(log N)
Space Complexity	O(1)

Tips & Tricks

- Use Linear Search when $N \leq 10^6$ or array is unsorted. $O(N)$ is fine.
- Use Binary Search when $N > 10^6$ AND array is sorted. $O(\log N)$ is crucial.
- Lower Bound: first index where $a[i] \geq \text{key}$. Frequency of key = upperBound - lowerBound.
- Binary search works on any monotonic function (binary search on answer).
- Always use $\text{mid} = \text{lo} + (\text{hi} - \text{lo}) / 2$ to avoid integer overflow.

6. Linear Search

Problem 6.1 — Find Key in Unsorted Array

Given an unsorted array of N integers and a key, return the first index where key is found, or -1.

```
#include <stdio.h>
int linearSearch(int a[], int n, int key)
{
    int i;
    for (i = 0; i < n; i++)
    {
        if (a[i] == key) return i;
    }
    return -1;
}
int main()
{
    int n, key, i;
    scanf("%d", &n);
    int a[n];
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
}
```

```
scanf("%d", &key);
printf("%d\n", linearSearch(a, n, key));
return 0;
}
```

Time Complexity	O(N)
Space Complexity	O(1)

7. Sorting Algorithms

Problem 7.1 — Merge Sort

Given an array of N integers, sort it in ascending order using Merge Sort.

```
#include <stdio.h>
void merge(int a[], int l, int m, int r)
{
    int n1=m-l+1, n2=r-m, L[n1], R[n2], i, j, k;
    for(i=0; i<n1; i++) L[i]=a[l+i];
    for(j=0; j<n2; j++) R[j]=a[m+1+j];
    i=0; j=0; k=l;
    while(i<n1 && j<n2)
    {
        if(L[i]<=R[j]) a[k++]=L[i++];
        else a[k++]=R[j++];
    }
    while(i<n1) a[k++]=L[i++];
    while(j<n2) a[k++]=R[j++];
}
void mergeSort(int a[], int l, int r)
{
    if(l<r)
    {
        int m=l+(r-l)/2;
        mergeSort(a, l, m);
        mergeSort(a, m+1, r);
        merge(a, l, m, r);
    }
}
int main()
{
    int n, i;
    scanf("%d", &n);
    int a[n];
    for(i=0; i<n; i++) scanf("%d", &a[i]);
    mergeSort(a, 0, n-1);
    for(i=0; i<n; i++) printf("%d ", a[i]);
    return 0;
}
```

Time Complexity	$O(N \log N)$ — all cases
Space Complexity	$O(N)$

Problem 7.2 — Quick Sort

Given an array of N integers, sort it in ascending order using Quick Sort.

```
#include <stdio.h>
void swap(int *a, int *b) { int t=*a; *a=*b; *b=t; }
int partition(int a[], int lo, int hi)
{
    int pivot=a[hi], i=lo-1, j;
    for(j=lo; j<hi; j++)
    {
        if(a[j]<=pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i+1], &a[hi]);
    return i+1;
}
void quickSort(int a[], int lo, int hi)
{
    if(lo<hi)
    {
        int pi=partition(a, lo, hi);
        quickSort(a, lo, pi-1);
        quickSort(a, pi+1, hi);
    }
}
int main()
{
    int n, i;
    scanf("%d", &n);
    int a[n];
    for(i=0; i<n; i++) scanf("%d", &a[i]);
    quickSort(a, 0, n-1);
    for(i=0; i<n; i++) printf("%d ", a[i]);
    return 0;
}
```

Time Complexity	$O(N \log N)$ avg, $O(N^2)$ worst
Space Complexity	$O(\log N)$

Tips & Tricks

→ Use Merge Sort when stability is needed or worst-case $O(N \log N)$ is required.
 → Use Quick Sort for in-place, cache-friendly sorting on arrays.

→ For $N \leq 10^5$ random data: Quick Sort is usually faster.
 For $N > 10^6$ or sorted data: Merge Sort.
 → Avoid Quick Sort worst-case $O(N^2)$: use random pivot or median-of-three.

8. Greedy Algorithms

Problem 8.1 — Coin Change (Greedy)

Given coin denominations $\{1, 5, 10, 25, 50, 100\}$ and an amount, find the minimum number of coins. (Works only for canonical coin systems.)

```
#include <stdio.h>
int main()
{
    int coins[]={1, 5, 10, 25, 50, 100};
    int n=6, amount, i, count=0;
    scanf("%d", &amount);
    for(i=n-1; i>=0 && amount>0; i--)
    {
        count += amount / coins[i];
        amount %= coins[i];
    }
    printf("%d\n", count);
    return 0;
}
```

Time Complexity	$O(D)$ where D = denominations
Space Complexity	$O(D)$

Problem 8.2 — Fractional Knapsack

Given N items with weights and values and capacity W , maximize total value (fractions allowed).

```
#include <stdio.h>
typedef struct { int weight, value; double ratio; } Item;
void sort(Item it[], int n)
{
    int i, j; Item tmp;
    for(i=0; i<n-1; i++)
        for(j=0; j<n-i-1; j++)
            if(it[j].ratio < it[j+1].ratio)
                { tmp=it[j]; it[j]=it[j+1]; it[j+1]=tmp; }
}
int main()
{
    int n, W, i;
    scanf("%d %d", &n, &W);
```

```

Item it[n];
for(i=0;i<n;i++)
{
    scanf("%d %d",&it[i].value,&it[i].weight);
    it[i].ratio=(double)it[i].value/it[i].weight;
}
sort(it,n);
double total=0.0;
for(i=0;i<n;i++)
{
    if(W>=it[i].weight)
    { total+=it[i].value; W-=it[i].weight; }
    else { total+=it[i].ratio*W; break; }
}
printf("%.2f\n",total);
return 0;
}

```

Time Complexity	O(N log N)
Space Complexity	O(N)

Problem 8.3 — Activity Selection

Given N activities with start and end times, select the maximum number of non-overlapping activities.

```

#include <stdio.h>
typedef struct { int s, e; } Act;
void sort(Act a[], int n)
{
    int i, j; Act tmp;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(a[j].e > a[j+1].e)
                { tmp=a[j]; a[j]=a[j+1]; a[j+1]=tmp; }
}
int main()
{
    int n, i, count, last;
    scanf("%d",&n);
    Act a[n];
    for(i=0;i<n;i++) scanf("%d %d",&a[i].s,&a[i].e);
    sort(a,n);
    count=1; last=a[0].e;
    for(i=1;i<n;i++)
    {
        if(a[i].s >= last)
            { count++; last=a[i].e; }
    }
    printf("%d\n",count);
    return 0;
}

```

Time Complexity	O(N log N)
Space Complexity	O(N)

Problem 8.4 — Job Sequencing with Deadlines

Given N jobs with deadlines and profits, schedule jobs to maximize total profit. Each job takes 1 unit of time.

```

#include <stdio.h>
typedef struct { int id, dl, pr; } Job;
void sort(Job j[], int n)
{
    int i, k; Job tmp;
    for(i=0;i<n-1;i++)
        for(k=0;k<n-i-1;k++)
            if(j[k].pr < j[k+1].pr)
                { tmp=j[k]; j[k]=j[k+1]; j[k+1]=tmp; }
}
int main()
{
    int n, i, t, maxDL=0, profit=0, cnt=0;
    scanf("%d",&n);
    Job j[n];
    for(i=0;i<n;i++)
    {
        scanf("%d %d %d",&j[i].id,&j[i].dl,&j[i].pr);
        if(j[i].dl > maxDL) maxDL=j[i].dl;
    }
    sort(j,n);
    int slot[maxDL+1];
    for(i=0;i<=maxDL;i++) slot[i]=-1;
    for(i=0;i<n;i++)
    {
        for(t=j[i].dl;t>=1;t--)
        {
            if(slot[t]==-1)
            {
                slot[t]=j[i].id;
                profit+=j[i].pr;
                cnt++;
                break;
            }
        }
    }
    printf("Jobs=%d Profit=%d\n",cnt,profit);
    return 0;
}

```

Time Complexity	O(N ²) or O(N log N) with DSU
------------------------	---

Space Complexity	O(N)
-------------------------	------

Problem 8.5 — Minimum Spanning Tree (Kruskal's)

Given a weighted undirected graph with N nodes and E edges, find the MST cost using Kruskal's algorithm.

```

#include <stdio.h>
typedef struct { int u, v, w; } Edge;
int parent[1001], rnk[1001];
int find(int x)
{
    if(parent[x]!=x) parent[x]=find(parent[x]);
    return parent[x];
}
void unite(int a, int b)
{
    a=find(a); b=find(b);
    if(rnk[a]<rnk[b]) { int t=a;a=b;b=t; }
    parent[b]=a;
    if(rnk[a]==rnk[b]) rnk[a]++;
}
void sort(Edge e[], int n)
{
    int i,j; Edge tmp;
    for(i=0;i<n-1;i++)
        for(j=0;j<n-i-1;j++)
            if(e[j].w>e[j+1].w)
                { tmp=e[j]; e[j]=e[j+1]; e[j+1]=tmp; }
}
int main()
{
    int n, m, i, cost=0, cnt=0;
    scanf("%d %d",&n,&m);
    Edge e[m];
    for(i=0;i<m;i++) scanf("%d %d %d",&e[i].u,&e[i].v,&e[i].w);
    for(i=0;i<n;i++) { parent[i]=i; rnk[i]=0; }
    sort(e,m);
    for(i=0;i<m && cnt<n-1;i++)
    {
        if(find(e[i].u)!=find(e[i].v))
            { unite(e[i].u,e[i].v); cost+=e[i].w; cnt++; }
    }
    printf("%d\n",cost);
    return 0;
}

```

Time Complexity	O(E log E)
Space Complexity	O(V + E)

Tips & Tricks

- Use Greedy when local optimal leads to global optimum (activity selection, Huffman, fractional knapsack).
- Greedy FAILS for 0/1 Knapsack and Coin Change with arbitrary denominations — use DP.
- Greedy vs DP: partial items => Greedy. Whole items => DP (0/1 Knapsack).
- Activity Selection: always sort by end time, pick earliest-ending non-overlapping activity.
- Kruskal's MST: sort edges by weight, use Union-Find to avoid cycles.
- Job Scheduling: sort by profit descending, fill latest available slot before deadline.

9. Dynamic Programming (DP)

Problem 9.1 — Coin Change (DP — Minimum Coins)

Given N coin denominations and an amount, find the minimum coins to make the amount. Output -1 if not possible.

```
#include <stdio.h>
#define INF 1000000000
int main()
{
    int n, amount, i, j;
    scanf("%d %d", &n, &amount);
    int coins[n];
    for(i=0; i<n; i++) scanf("%d", &coins[i]);
    int dp[amount+1];
    for(i=0; i<=amount; i++) dp[i]=INF;
    dp[0]=0;
    for(i=1; i<=amount; i++)
    {
        for(j=0; j<n; j++)
        {
            if(coins[j]<=i && dp[i-coins[j]]!=INF)
                if(dp[i-coins[j]]+1 < dp[i])
                    dp[i]=dp[i-coins[j]]+1;
        }
    }
    printf("%d\n", dp[amount]==INF ? -1 : dp[amount]);
    return 0;
}
```

Time Complexity	$O(N \times \text{Amount})$
-----------------	-----------------------------

Space Complexity	$O(\text{Amount})$
------------------	--------------------

Problem 9.2 — 0/1 Knapsack (DP)

Given N items with weights and values, and capacity W , find the maximum value achievable (each item used at most once).

```
#include <stdio.h>
int max(int a, int b) { return a>b?a:b; }
int main()
{
    int n, W, i, w;
    scanf("%d %d", &n, &W);
    int wt[n], val[n];
    for(i=0; i<n; i++) scanf("%d %d", &wt[i], &val[i]);
    int dp[n+1][W+1];
    for(i=0; i<=n; i++)
    {
        for(w=0; w<=W; w++)
        {
            if(i==0||w==0) dp[i][w]=0;
            else if(wt[i-1]<=w)
                dp[i][w]=max(val[i-1]+dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else dp[i][w]=dp[i-1][w];
        }
    }
    printf("%d\n", dp[n][W]);
    return 0;
}
```

Time Complexity	$O(N \times W)$
-----------------	-----------------

Space Complexity	$O(N \times W)$
------------------	-----------------

Problem 9.3 — Longest Common Subsequence (LCS)

Given two strings of lengths M and N , find the length of their Longest Common Subsequence.

```
#include <stdio.h>
#include <string.h>
int max(int a, int b) { return a>b?a:b; }
int main()
{
    char s1[1001], s2[1001];
    scanf("%s %s", s1, s2);
    int m=strlen(s1), n=strlen(s2), i, j;
    int dp[m+1][n+1];
    for(i=0; i<=m; i++)
    {
        for(j=0; j<=n; j++)
        {
            if(i==0||j==0) dp[i][j]=0;
            else if(s1[i-1]==s2[j-1])
                dp[i][j]=dp[i-1][j-1]+1;
            else
                dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
        }
    }
    printf("%d\n", dp[m][n]);
    return 0;
}
```

Loop & Logic

```
dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
}
}
printf("%d\n", dp[m][n]);
return 0;
}
```

Time Complexity	$O(M \times N)$
-----------------	-----------------

Space Complexity	$O(M \times N)$
------------------	-----------------

Problem 9.4 — Longest Increasing Subsequence (LIS)

Given an array of N integers, find the length of the longest strictly increasing subsequence.

```
#include <stdio.h>
int max(int a, int b) { return a>b?a:b; }
int main()
{
    int n, i, j, ans=1;
    scanf("%d", &n);
    int a[n], dp[n];
    for(i=0; i<n; i++) scanf("%d", &a[i]);
    for(i=0; i<n; i++) dp[i]=1;
    for(i=1; i<n; i++)
    {
        for(j=0; j<i; j++)
        {
            if(a[j]<a[i] && dp[j]+1>dp[i])
                dp[i]=dp[j]+1;
        }
        if(dp[i]>ans) ans=dp[i];
    }
    printf("%d\n", ans);
    return 0;
}
```

Time Complexity	$O(N^2)$ DP / $O(N \log N)$ with patience sort
-----------------	--

Space Complexity	$O(N)$
------------------	--------

Problem 9.5 — Rod Cutting (DP)

Given a rod of length N and a price array where price[i] is the price of a rod of length $i+1$, find the maximum revenue.

```
#include <stdio.h>
int max(int a, int b) { return a>b?a:b; }
int main()
{
    // ...
}
```

```

int n, i, j;
scanf("%d",&n);
int p[n+1], dp[n+1];
p[0]=0;
for(i=1;i<=n;i++) scanf("%d",&p[i]);
for(i=0;i<=n;i++) dp[i]=0;
for(i=1;i<=n;i++)
{
    for(j=1;j<=i;j++)
    {
        if(p[j]+dp[i-j] > dp[i])
            dp[i]=p[j]+dp[i-j];
    }
}
printf("%d\n",dp[n]);
return 0;
}

```

Time Complexity	$O(N^2)$
Space Complexity	$O(N)$

Tips & Tricks

→ Use DP when: overlapping subproblems + optimal substructure exist (LCS, LIS, Knapsack, Coin Change).
 → Greedy vs DP: Greedy makes one pass, no revisiting. DP fills a table using prior results.
 → 0/1 Knapsack: each item once $\Rightarrow$ $dp[i][w]$. Unbounded: items reusable $\Rightarrow$ 1D $dp[w]$.
 → LCS classic use: diff tools, DNA comparison. LIS use: patience sorting, stock markets.
 → Rod Cutting = Unbounded Knapsack: each piece can be reused; loop j from 1 to i .
 → Space optimization: 0/1 Knapsack 2D $dp[N][W] \Rightarrow$ 1D $dp[W]$ iterating w in reverse.
 → Memoization (top-down) vs Tabulation (bottom-up): tabulation is usually faster.

10. Breadth-First Search (BFS)

Problem 10.1 — BFS Traversal (Adjacency Matrix)

Given an undirected graph with N nodes and E edges, perform BFS traversal starting from node 0.

```

#include <stdio.h>
#define MAX 100
int adj[MAX][MAX], visited[MAX];
int queue[MAX], front=0, rear=0;

```

```

void bfs(int start, int n)
{
    visited[start]=1;
    queue[rear++]=start;
    while(front<rear)
    {
        int node=queue[front++], i;
        printf("%d ",node);
        for(i=0;i<n;i++)
        {
            if(adj[node][i] && !visited[i])
            { visited[i]=1; queue[rear++]=i; }
        }
    }
}

int main()
{
    int n, e, u, v, i;
    scanf("%d %d",&n,&e);
    for(i=0;i<e;i++)
    {
        scanf("%d %d",&u,&v);
        adj[u][v]=1; adj[v][u]=1;
    }
    bfs(0,n);
    return 0;
}

```

Time Complexity	$O(V^2)$ adj matrix / $O(V+E)$ adj list
Space Complexity	$O(V)$

11. Depth-First Search (DFS)

Problem 11.1 — DFS Traversal (Adjacency Matrix)

Given an undirected graph with N nodes and E edges, perform DFS traversal starting from node 0.

```

#include <stdio.h>
#define MAX 100
int adj[MAX][MAX], visited[MAX];
void dfs(int node, int n)
{
    visited[node]=1;
    printf("%d ",node);
    int i;
    for(i=0;i<n;i++)
    {
        if(adj[node][i] && !visited[i]) dfs(i,n);
    }
}

```

```

int main()
{
    int n, e, u, v, i;
    scanf("%d %d",&n,&e);
    for(i=0;i<e;i++)
    {
        scanf("%d %d",&u,&v);
        adj[u][v]=1; adj[v][u]=1;
    }
    dfs(0,n);
    return 0;
}

```

Time Complexity	$O(V^2)$ adj matrix / $O(V+E)$ adj list
Space Complexity	$O(V)$

Tips & Tricks

→ BFS: explores level by level (Queue). Best for shortest path in unweighted graphs.
 → DFS: explores depth first (Stack/Recursion). Best for cycle detection, topological sort, SCCs.
 → BFS gives shortest path (fewest edges) in unweighted graphs. For weighted graphs use Dijkstra.
 → BFS for: multi-source shortest path, bipartite check. DFS for: flood fill, maze solving.

12. Shortest Path Algorithms

Problem 12.1 — Dijkstra's Algorithm

Given a weighted undirected graph with N nodes and E edges (non-negative weights), find shortest distances from source node 0.

```

#include <stdio.h>
#define MAX 100
#define INF 1000000000
int n, e, adj[MAX][MAX], dist[MAX], vis[MAX];
int minDist()
{
    int mn=INF, idx=-1, i;
    for(i=0;i<n;i++)
    {
        if(!vis[i] && dist[i]<mn) { mn=dist[i]; idx=i; }
    }
    return idx;
}
void dijkstra(int src)
{
    int i, u, v;
}

```

```

for(i=0;i<n;i++) { dist[i]=INF; vis[i]=0; }
dist[src]=0;
for(i=0;i<n-1;i++)
{
    u=minDist();
    if(u==-1) break;
    vis[u]=1;
    for(v=0;v<n;v++)
    {
        if(!vis[v] && adj[u][v]
        && dist[u]+adj[u][v]<dist[v])
            dist[v]=dist[u]+adj[u][v];
    }
}
}
int main()
{
    int u, v, w, i;
    scanf("%d %d", &n, &e);
    for(i=0; i<e; i++)
    {
        scanf("%d %d %d", &u, &v, &w);
        adj[u][v]=w; adj[v][u]=w;
    }
    dijkstra(0);
    for(i=0; i<n; i++) printf("0->%d: %d\n", i, dist[i]);
    return 0;
}

```

Time Complexity	$O(V^2)$ simple / $O((V+E) \log V)$ with PQ
Space Complexity	$O(V)$

Problem 12.2 — Bellman-Ford Algorithm

Given a directed graph with N nodes and E edges (may have negative weights), find shortest distances from source 0. Detect negative cycles.

```

#include <stdio.h>
#define MAX 1000
#define INF 1000000000
typedef struct { int u, v, w; } Edge;
Edge edges[MAX];
int dist[MAX];
int main()
{
    int n, e, i, j;
    scanf("%d %d", &n, &e);
    for(i=0; i<e; i++)

```

```

    scanf("%d %d
    %d", &edges[i].u, &edges[i].v, &edges[i].w);
    for(i=0; i<n; i++) dist[i]=INF;
    dist[0]=0;
    for(i=0; i<n-1; i++)
    {
        for(j=0; j<e; j++)
        {
            int u=edges[j].u, v=edges[j].v, w=edges[j].w;
            if(dist[u]!=INF && dist[u]+w<dist[v])
                dist[v]=dist[u]+w;
        }
    }
    int neg=0;
    for(j=0; j<e; j++)
    {
        int u=edges[j].u, v=edges[j].v, w=edges[j].w;
        if(dist[u]!=INF && dist[u]+w<dist[v]) { neg=1; break; }
    }
    if(neg) printf("Negative cycle\n");
    else for(i=0; i<n; i++) printf("0->%d: %d\n", i, dist[i]);
    return 0;
}

```

Time Complexity	$O(V \times E)$
Space Complexity	$O(V + E)$

Tips & Tricks

→ Dijkstra: non-negative weights only. $O(V^2)$ simple / $O((V+E) \log V)$ with priority queue.
 → Bellman-Ford: handles negative weights, detects negative cycles. $O(VE)$ — slower but versatile.
 → Floyd-Warshall: all-pairs shortest path, $O(V^3)$ — use when finding shortest between every pair.
 → Use Dijkstra for single-source non-negative graphs (road networks, game maps).
 → Use Bellman-Ford for graphs with negative edges (currency exchange, network arbitrage).
 → Dense graphs: adjacency matrix Dijkstra $O(V^2)$. Sparse graphs: min-heap $O((V+E) \log V)$.